TECHNICAL TRAINING

SRE in Practice

Turning reliability into a number the business agrees to, a budget the team can spend, and a process that survives the bad night.

Vivekanand Pujar | @VivekanandPujar

Consultant-led workshop · SREs, platform engineers, service owners and engineering leadership

Six sessions · Service levels, budgets, releases, signals, postmortems and incident response

SESSION MAP

Agenda

01 **How reliable is reliable enough**

SLIs, SLOs and SLAs: what users feel, what the team targets, what the contract prices.

02 **Error budgets**

The gap between the objective and perfection, spent on change and defended by policy.

03 **Release engineering and environments**

Why most outages begin with a change, and how the pipeline limits the damage.

04 **The four golden signals**

Latency, traffic, errors and saturation, and the traps inside each one.

05 **Blameless postmortems**

Asking why the system allowed the mistake, and closing the loop with owned actions.

06 **Emergency response and runbooks**

Incident command roles, mitigation before diagnosis, and runbooks that work at 3 a.m.

Vivekanand Pujar | @VivekanandPujar

THE ARGUMENT

What this course is trying to establish

> Users cannot tell 100% from 99.99%, but the business pays heavily for every nine in between. Reliability is a feature with a price: agree the target with the people who pay, spend what is left over on change, and build the process that recovers when the target is missed.

Set the target from the user, not the server

A dashboard of green hosts can sit beside a checkout that fails for one customer in fifty. The measure starts where the user's request starts.

Unspent reliability is wasted velocity

Every minute of availability above the objective is a release that was not shipped. The budget makes the trade explicit and removes the argument.

Incidents are paid for once; lessons are free

An outage has already cost what it cost. A blameless review and a runbook are how the team stops paying for the same failure twice.

Vivekanand Pujar | @VivekanandPujar

SESSION 01

Three numbers, three owners: what users feel, what we aim for, what we owe

SLI Service level indicator

What does the user experience?

A measured ratio: good events divided by valid events, over a window.

EXAMPLE

Checkouts completing in under 400 ms, as a share of all checkouts.

OWNED BY

The service team, from telemetry

SLO Service level objective

How good, and over what window?

A target for the SLI, set below what the system achieves on a good day.

EXAMPLE

99.9% of checkouts succeed in under 400 ms, over 28 days.

OWNED BY

Product and engineering, jointly

SLA Service level agreement

What happens if we miss?

A contract with credits or refunds attached. Always looser than the SLO.

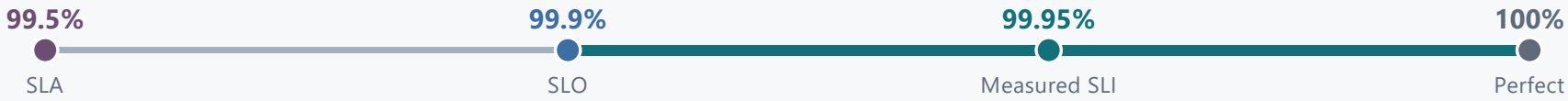
EXAMPLE

Below 99.5% monthly availability, customers receive a 10% credit.

OWNED BY

Legal and commercial, with engineering

WHERE THE NUMBERS SIT FOR ONE CHECKOUT SERVICE, 28-DAY WINDOW · NOT TO SCALE



Margin before credits are owed

Error budget: 0.1% of checkouts may fail; spend it on change

Vivekanand Pujar | @VivekanandPujar

SESSION 01

A useful SLI is a ratio of good events, measured as close to the user as possible

THE SHAPE OF EVERY SLI

SLI = good events ÷ valid events

A ratio from 0% to 100% makes every SLI comparable, and turns the objective into a count of events the service may get wrong.

WHERE TO MEASURE: TRUTH RISES TOWARD THE USER

Server logs → Load balancer → Synthetic probe → **Real client**

Server-side is cheap and clean but misses DNS, network and client failures. Client-side sees everything, including noise the team cannot fix.

SLI TYPE	A GOOD EVENT IS…	FITS	THE TRAP
Availability	A request that returns a non-5xx response	Request-driven APIs	Counting 4xx as good hides a broken client release
Latency	A request answered faster than a threshold	Interactive paths	An average of 120 ms can hide a p99 of 3 s
Correctness	A response with the right content	Search, pricing, payments	A fast 200 carrying a wrong price is still counted good
Freshness	Data updated within a stated age	Pipelines, caches, feeds	Measuring job success, not the age of what users read
Durability	A stored object still readable later	Storage, backups	Untested restores: the backup exists, the data does not

Vivekanand Pujar | @VivekanandPujar

SESSION 01

Each extra nine cuts allowed downtime tenfold, and changes how the team has to operate

ALLOWED DOWNTIME AT EACH OBJECTIVE, TIME-BASED AVAILABILITY

OBJECTIVE	PER 30 DAYS	PER YEAR	WHAT IT TAKES TO HOLD
99%	7.2 hours	3.65 days	Business hours support; manual restarts
99.9%	43 minutes	8.8 hours	On-call paging; rollback within minutes
99.95%	22 minutes	4.4 hours	Automated rollback; redundant zones
99.99%	4.3 minutes	53 minutes	No human in the loop; automated failover
99.999%	26 seconds	5.3 minutes	Multi-region active-active; years of investment

At four nines the monthly allowance is shorter than the time it takes to page a human.

DEPENDENCIES MULTIPLY

A service can be no more available than the product of the hard dependencies it calls in series.

3 dependencies at 99.9%

0.999^3 **99.70%**

5 dependencies at 99.99%

0.9999^5 **99.95%**

Before promising a nine, list what sits in the request path. Caches, fallbacks and asynchronous calls take a dependency out of the product.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

An error budget turns reliability into a quantity the team can spend

THE ARITHMETIC, FOR ONE CHECKOUT SERVICE

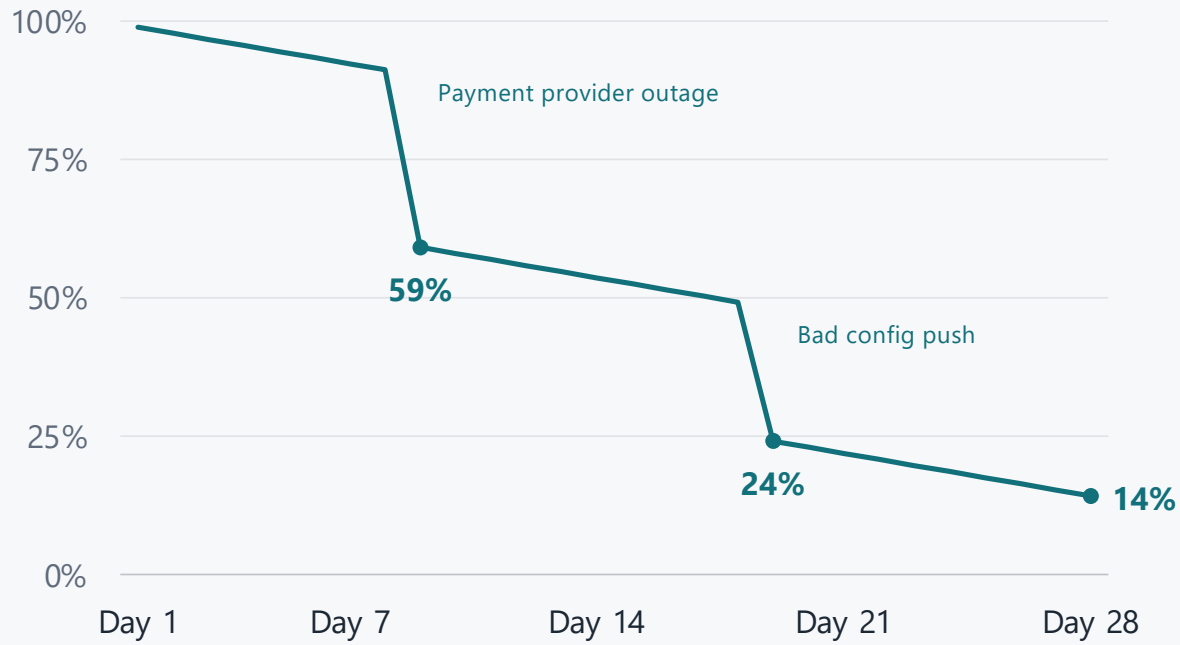
SLO	**99.9% over 28 days**
Checkouts in the window	**12 million**
Failures allowed	**12,000**
Or, as a full outage	**40 minutes**

WHAT SPENDS IT

- Releases and config changes
- Experiments and migrations
- Incidents and dependency failures
- Planned maintenance

The budget is not a target to miss. It is permission to take risk, sized in advance.

ERROR BUDGET REMAINING, CHECKOUT SERVICE, DAYS 1–28 OF THE WINDOW, %



Illustrative window. Background errors burn about 1% a day; two incidents consumed over half the budget.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

The budget policy decides in advance what happens when the budget runs low

Without a written policy, every freeze is negotiated mid-incident by the people with the least sleep.

MORE THAN 50% LEFT	25% TO 50% LEFT	UNDER 25% LEFT	BUDGET EXHAUSTED
Ship normally	**Tighten**	**Slow down**	**Freeze**
Releases and experiments proceed at the usual pace.	Risky changes need a rollback plan and a second reviewer.	Reliability work moves ahead of features in the next sprint.	Only fixes and security patches ship until the window recovers.

THE INTUITIVE DEFAULT

Treat the SLO as a dashboard and decide case by case. The loudest voice wins, the freeze arrives late, and product learns that reliability is an SRE opinion rather than a shared number.

THE PRACTICE

- Product, engineering and SRE sign the policy before it is needed.
- One incident burning over 20% of the budget gets a postmortem.
- Disputes go to one named executive, not to a meeting.
- An SLO never breached is set too low; revisit it each quarter.

Vivekanand Pujar | @VivekanandPujar

SESSION 02

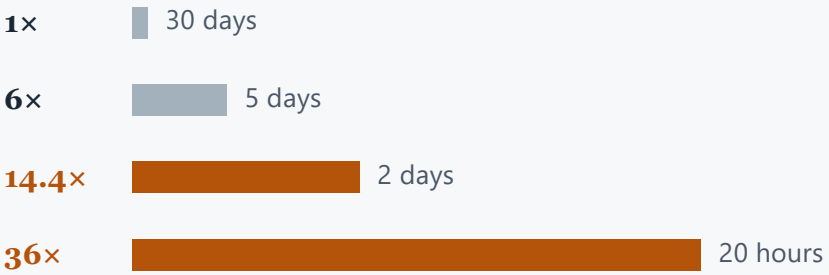
Alert on how fast the budget is burning, not on every blip in the error rate

BURN RATE: THE SPEED OF SPENDING

burn rate = error rate ÷ (1 − SLO)

On a 99.9% SLO, a 1.44% error rate is a burn rate of 14.4: the 30-day budget is gone in about two days.

TIME UNTIL A 30-DAY BUDGET IS EXHAUSTED



A fixed threshold such as "errors above 0.1% for five minutes" pages for a blip that costs 0.01% of the budget, and stays silent through a slow leak that drains it over a week.

MULTI-WINDOW ALERTS FOR A 99.9% SLO, 30-DAY WINDOW

RESPONSE	BURN RATE	WINDOWS	BUDGET SPENT
Page	14.4×	1 hour / 5 min	2%
Page	6×	6 hours / 30 min	5%
Ticket	1×	3 days / 6 hours	10%

WHY TWO WINDOWS

The long window proves the burn is sustained and significant, so a 30-second spike never wakes anyone.

The short window must also be burning, so the alert clears within minutes of recovery instead of an hour later.

Source: Google, The Site Reliability Workbook, "Alerting on SLOs" (2018).

Vivekanand Pujar | @VivekanandPujar

SESSION 03

Roughly 70% of outages begin with a change, so the release path is a reliability control



Every stage either promotes the same artifact forward or sends it back; nothing is rebuilt between stages.

WHY THE PIPELINE CARRIES THE RELIABILITY WORK

Build once, promote the artifact

The binary that passed staging is the binary in production. A rebuild per environment ships something nobody tested.

Configuration is code

A flag flip or a quota change reaches every server at once. Config pushes need review, staging and a canary like any binary.

Rollback before root cause

The first response to a bad release is to reverse it. Diagnosis happens after users are safe, not while they wait.

THE OPERATING CONTRACT

Small batches — one change per release, so a regression has one suspect.

Deploy is not release — feature flags ship dark code and expose it per tenant or per percentage.

Automated gates — a release that breaches the canary SLO reverses itself, without a human.

Source: Google, Site Reliability Engineering (2016): roughly 70% of outages are due to changes in a live system.

Vivekanand Pujar | @VivekanandPujar

SESSION 03

Environments should differ in data and scale, never in how they are built

	DEVELOPMENT	STAGING	PRODUCTION
Purpose	Fast feedback for one engineer	Prove the release before users see it	Serve users within the SLO
Data	Synthetic fixtures	Masked copy or generated at volume	Real, regulated, irreplaceable
Scale	One replica, local or ephemeral	Same topology, smaller footprint	Full capacity plus headroom
Changed by	The engineer, freely	The pipeline only	The pipeline only, behind gates
Built from	Same modules, same pipeline	Same modules, same pipeline	Same modules, same pipeline

WHERE PARITY BREAKS

- A hand-built staging that nobody can recreate
- Hotfixes applied in production and never committed
- Staging shared by twelve teams, so it is never green

THE PRACTICE

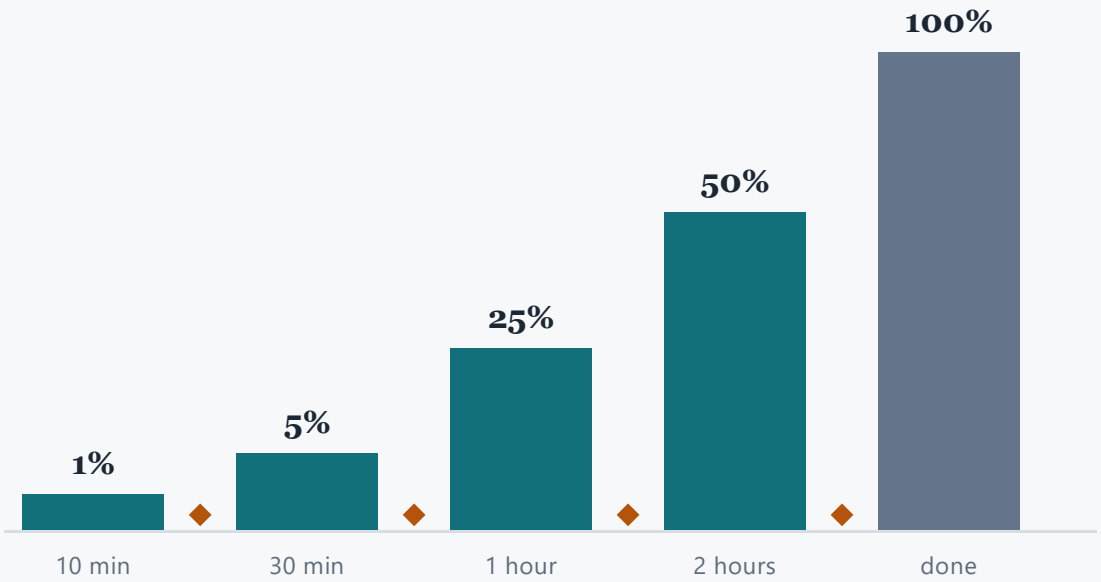
- Every environment comes from one set of IaC modules
- Ephemeral preview environments per pull request
- GitOps drift detection alerts on hand edits in production

Vivekanand Pujar | @VivekanandPujar

SESSION 03

A canary limits a bad release to a slice of users and a few minutes

PROGRESSIVE ROLLOUT, SHARE OF TRAFFIC ON THE NEW VERSION · NOT TO SCALE



◆ SLO gate: canary compared with a baseline cohort on errors, p99 latency and saturation

Gate fails → traffic returns to the previous version, with no human in the loop.

BLAST RADIUS = TRAFFIC SHARE × TIME TO DETECT

A release that fails every request, on a 99.9% SLO with a 40-minute monthly budget:

100% of traffic, 10 minutes	**25% of budget**
1% of traffic, 10 minutes	**0.25% of budget**

PRACTICE

- Compare with a baseline started at the same moment, not with yesterday.
- Bake long enough to cross a cron cycle and a cache expiry.
- Keep a feature-flag kill switch for what the canary cannot see.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

Four signals cover almost everything a user can feel about a service

Latency

How long did it take?

Time to serve a request, split into successes and failures.

MEASURE

p50, p99 and p99.9 of request duration, per endpoint

THE TRAP

A fast error flatters the average; a timeout never reports.

Traffic

How much is being asked?

Demand on the system in its own units.

MEASURE

Requests per second, sessions, messages consumed, bytes

THE TRAP

A drop to zero looks calm; it can mean nobody can reach you.

Errors

How much is failing?

The rate of requests that fail, explicitly or by policy.

MEASURE

5xx rate, wrong content, responses over the latency SLO

THE TRAP

A 200 with an empty cart page is still an error to the user.

Saturation

How full is it?

How close the most constrained resource is to its limit.

MEASURE

CPU, memory, pool and queue occupancy, disk, file handles

THE TRAP

Utilisation at 70% can already mean queueing at the p99.

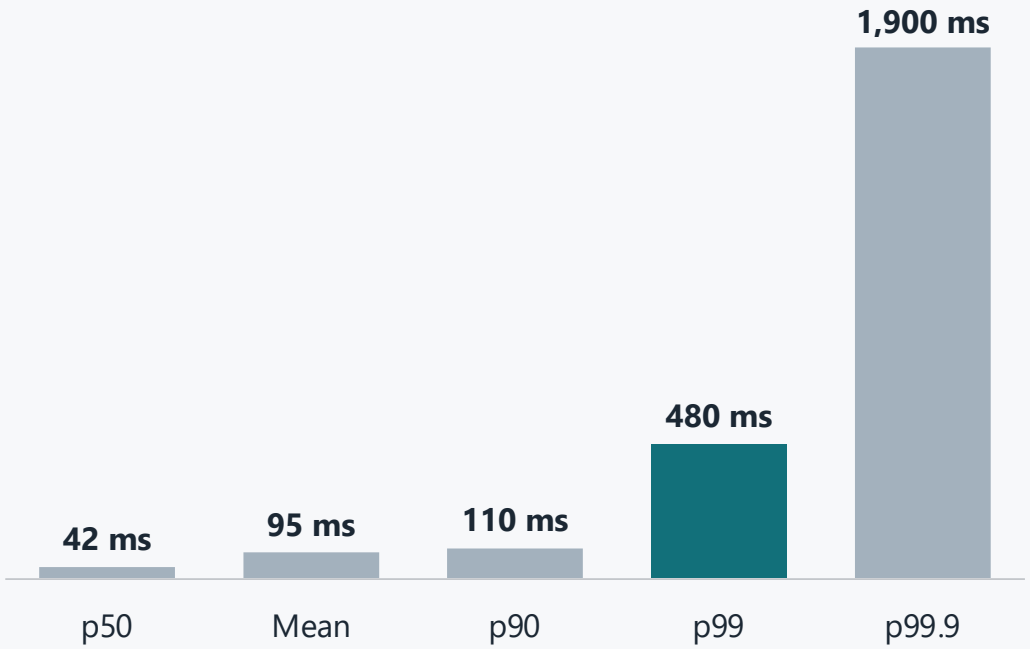
Page on the first three when they breach the SLO; watch saturation to see the next breach coming.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

The average reports 95 ms; one checkout in a hundred waits 480 ms or more

CHECKOUT API REQUEST LATENCY BY PERCENTILE, MS, ONE DAY (ILLUSTRATIVE)



The tail pulls the mean up, yet the mean still hides it. Set latency SLOs on percentiles.

FAN-OUT TURNS THE TAIL INTO THE MEDIAN

If one page calls N back ends in parallel, it waits for the slowest. The chance that at least one hits its p99 is $1 - 0.99^N$.

1 back end	1%
10 back ends	10%
100 back ends	63%

PRACTICE

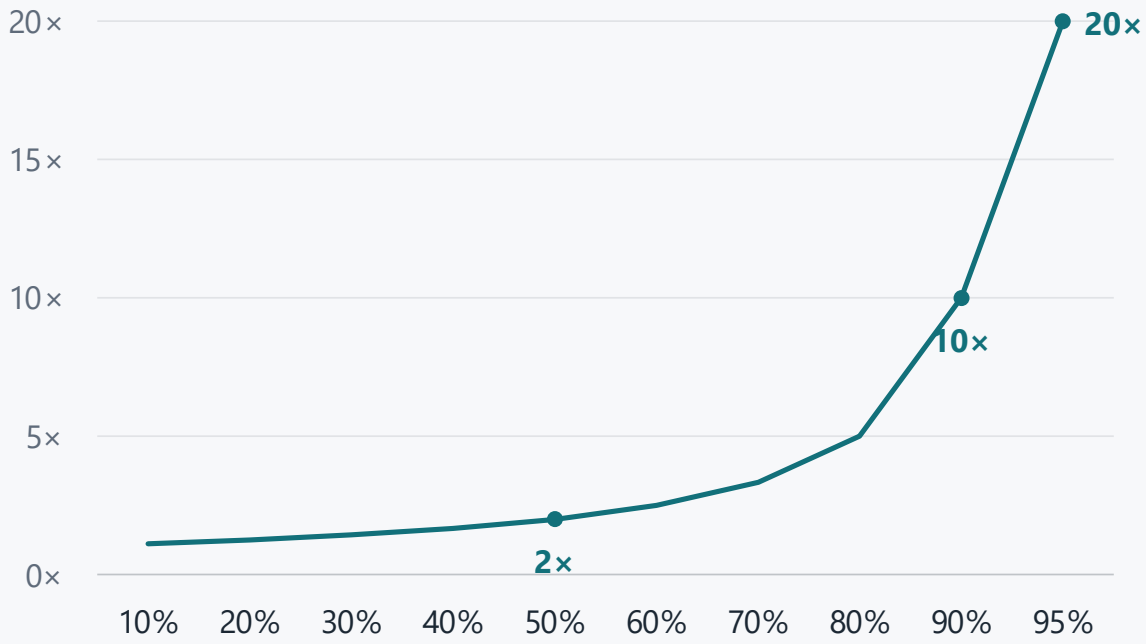
- Report p50, p99 and p99.9 per endpoint, never one blended number.
- Hedge or cancel slow calls; cap fan-out on the hot path.

Vivekanand Pujar | @VivekanandPujar

SESSION 04

Saturation warns before the other three: response time doubles at 50% busy and hits 10× at 90%

RESPONSE TIME RELATIVE TO AN IDLE SERVER, BY UTILISATION, × (SINGLE-QUEUE MODEL)



M/M/1 queue: response time = service time ÷ (1 − utilisation). Real systems differ in shape, not in direction.

WHY IT LEADS

Latency and errors report damage users have already felt. Saturation shows the queue forming, while there is still time to add capacity or shed load.

WATCH THE TIGHTEST RESOURCE

Pools threads, connections, worker slots

Queues depth and age of the oldest item

Finite stores disk, inodes, quota: time until full

Plan capacity for the headroom the p99 needs, not for the average utilisation that looks efficient.

Vivekanand Pujar | @VivekanandPujar

SESSION 05

A blameless postmortem asks why the system allowed the mistake, not who made it

ONE INCIDENT, TWO REVIEWS

The incident: an engineer ran a cleanup script against production instead of staging. Checkout failed for 40 minutes.

THE BLAME QUESTION

Who ran the script?

WHAT FOLLOWS

- The engineer is retrained or disciplined.
- The next engineer hides the near miss.
- The script can still reach production tomorrow.

THE SYSTEMS QUESTION

Why could the script reach production?

WHAT FOLLOWS

- Credentials were shared across environments → **separate them**
- The tool had no environment guard → **require an explicit target**
- Nobody noticed for 40 minutes → **alert on deletion rate**

THE WORKING ASSUMPTIONS

Everyone acted reasonably given what they knew at the time. If one person could make the mistake, the next person will.

Human error is where the investigation starts, not where it ends. Fix the conditions and the error becomes impossible or harmless.

Vivekanand Pujar | @VivekanandPujar

SESSION 05

A postmortem is finished when every action has an owner and a date

WHEN TO WRITE ONE

- A user-visible SLO breach
- One incident burning over 20% of budget
- Any data loss
- A rollback or manual intervention
- A near miss worth sharing

WHAT IT CONTAINS

01 Summary and impact
users affected, duration, budget spent

02 Timeline
detection, escalation, mitigation, in UTC

03 Causes
the trigger, and the conditions that let it spread

04 Went well, went wrong, got lucky
luck is a risk nobody planned for

ACTION ITEMS, FROM THE CLEANUP-SCRIPT INCIDENT

TYPE	ACTION	OWNER	DUE
Prevent	Cleanup tool refuses to run without an explicit target environment	Platform team	2 weeks
Detect	Alert when row deletions exceed three times the daily baseline	SRE on-call	1 week
Mitigate	Drill point-in-time restore and publish the runbook	Data team	This quarter

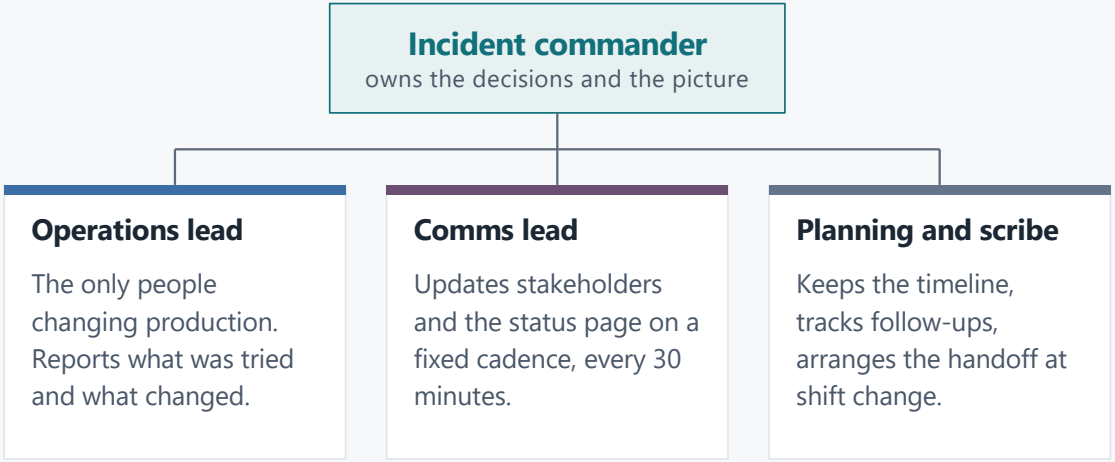
An action without an owner is a wish. Review open items weekly until closed.

Vivekanand Pujar | @VivekanandPujar

SESSION 06

Incident command separates the people fixing it from the person coordinating it

ROLES IN A MAJOR INCIDENT



WHAT THE COMMANDER DOES NOT DO

Type commands. Once the commander starts debugging, nobody holds the picture: who is doing what, what was tried, and when the next update is due.

PRINCIPLES THAT HOLD UNDER PRESSURE

Declare early.
Cancelling an incident costs minutes; declaring one late costs the first hour.

Mitigate, then diagnose.
Roll back, fail over or shed load first. Root cause can wait for the postmortem.

One commander at a time.
Handoffs are said aloud, "You now have command", and logged with the time.

Work in the open.
One channel, one call, one running log that anyone can read.

Vivekanand Pujar | @VivekanandPujar

SESSION 06

A runbook is written for a tired engineer at 3 a.m., and its best steps end up automated

ANATOMY, FOR THE ALERT CHECKOUTBUDGETBURNFAST

01 What fired
14.4× burn on checkout, 1-hour and 5-minute windows

→

02 What it means
2% of the monthly budget spent in the last hour

→

03 Check impact
Dashboard link; error share by region and payment provider

→

04 Mitigate
Roll back the last deploy, or fail over to the second provider

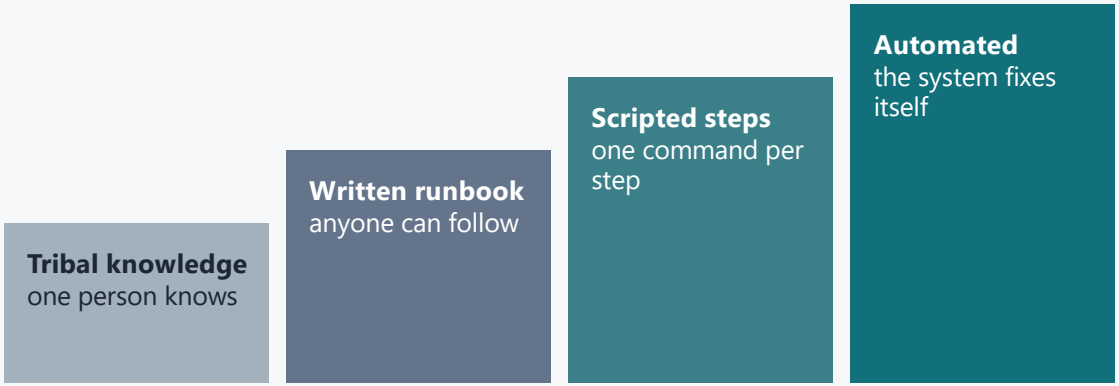
→

05 Escalate
Payments on-call after 15 minutes; open an incident

→

06 Verify
Burn under 1× for 30 minutes, then hand back and log

FROM MEMORY TO MACHINE



RULES THAT KEEP RUNBOOKS ALIVE

- Every page links to its runbook, or it does not page.
- Mitigation first; diagnosis steps come after users are safe.
- Each page carries an owner and a last-tested date.
- Game days exercise the runbook, not just the system.

Vivekanand Pujar | @VivekanandPujar

CLOSING

Ten questions for the next reliability review

SERVICE LEVELS

01 Which SLI would a user recognise as their own experience?

02 Who signed the SLO, and when was it last revisited?

03 How much budget is left in this window, and what spent it?

CHANGE

04 Which release in the last month reversed itself without a human?

05 Where does staging differ from production in how it is built?

SIGNALS

06 Which alert pages on a blip rather than a burn rate?

07 What is the tightest resource, and how much headroom does the p99 need?

INCIDENTS

08 Which postmortem action is overdue, and who owns it?

09 Which paging alert has no runbook, or one never tested?

10 Who is incident commander tonight, and do they know it?

Vivekanand Pujar | @VivekanandPujar